

MockAI: A Multi-Agent LLM Framework for Automated Technical Interview Assessment with Real-Time Gaze-Based Proctoring

1st Renuka N

Dept. of AI & ML

Kongu Engineering College
Perundurai, Erode, India
renuka66ksr@gmail.com

2nd Kannan N

Dept. of AI & DS

Kongu Engineering College
Perundurai, Erode, India
kannanmese@gmail.com

3rd Karthikeyan T A

Dept. of AI & DS

Kongu Engineering College
Perundurai, Erode, India
tak03793@gmail.com

4th Rithik R

Dept. of AI & ML

Kongu Engineering College
Perundurai, Erode, India
rithikyalini@gmail.com

5th Sanjay K

Dept. of AI & ML

Kongu Engineering College
Perundurai, Erode, India
ksanjayias@gmail.com

Abstract—Scaling up hiring is difficult. Just one college campus hiring event might lead to hundreds of candidates being selected, while the solutions available for engineering colleges and small companies are either costly (commercial online proctoring platforms) or inflexible (question banks with rules). In order to solve the above problem, we developed the MockAI solution, which uses an eight-agent architecture to conduct a three-round mock interview consisting of aptitude MCQ test, technical interview conducted by AI, and faculty-led behavioral interview.

The LLM agents operate via LangChain using the Groq API (openai/gpt-oss-120b), which took 480 ms per response on average in our tests, fast enough for natural conversations. The gaze proctoring operates alongside, on the same CPU, analyzing each webcam frame in 18 ms at the stream speed of 30 fps. Scores are obtained from the freeform output of LLMs via a four pattern cascade of regexes, falling back to linear mapping of the keyword-sentiment ratio to 25-45 in case all four patterns fail. We established that the scores of responses evaluated by human judges as good lie in the range 38-47/50, acceptable ones – in 28-37/50, poor – in 15-27/50. Accuracy of the proctoring falls off once the angle between the axis of the candidate’s face and vertical exceeds 30 degrees or the lighting becomes strongly asymmetric, a problem we openly acknowledge. The precise measurements of gaze accuracy and correlation between scores are to be done before the system can be released; these experiments are described in Section V.

Index Terms—multi-agent LLM; automated interview; gaze-based proctoring; LangChain; Groq API; OpenCV; Haar Cascade; recruitment automation; ConversationBufferMemory; regex score extraction; Flask; MongoDB Atlas; React

I. INTRODUCTION

Each year, engineering branches conduct placement drives, which end up giving hundreds of students technical interviews in a matter of days. Scheduling human interviewers consecutively is impractical, and the scores obtained are inconsistent, depending on the individual who conducts the interview.

In case the panel is well-trained and calibrated, the panelist on day three will ask different follow-up questions from the panelist on day one, especially when he or she is tired.

A common solution to this problem in most institutions is the creation of a question bank. This works, but it has a different set of problems in that the questions do not depend on what the student has just stated, making it impossible to determine whether the student answered the question correctly or was simply memorizing the questions. There are remote proctoring software tools available, but the most popular ones (Proctorio and ExamSoft) are too expensive.

LLMs shift the possibilities entirely. When you have a model capable of generating a question from the history of the entire conversation, the problem of follow-ups becomes solvable without needing human intervention [1]. The only problem left to solve now is how to integrate this with light proctoring and accurate scoring in a manner that works on existing college hardware. This is precisely what MockAI does.

The particular challenges that we wanted to overcome are:

- Need for context-awareness. Traditional systems generate every question separately, whereas our system is based on ConversationBufferMemory so that every prompt takes into account the entire conversation so far.
- Proctoring cost. We required a solution that worked on CPU, generated an alarm after 2.0 seconds of continuous deviation of gaze, and did not generate multiple alarms after a single gaze.
- Score extraction failure. A simple regex works poorly if the LLM starts writing a story instead of generating the number.
- Deployment challenges. Our solution needed to be deployed on a standard Intel i7 laptop without a GPU allocated in the cloud.

The novel contributions made by this research include:

- 1) An eight-agent deterministic LangChain pipeline that performs the whole aptitude-technical-behavior interview process sequence complete with role-based access control and session-level memory.
- 2) A CPU-only proctoring agent employing Haar Cascade face detection and pupil centroid detection using image moment analysis, operating on a frame-to-frame basis at 18 ms/30 fps and 2.0 second alert threshold with 5.0 seconds cooling period.
- 3) A six step contextual technical interview with ConversationBufferMemory preserving full dialogue history allowing for follow-up questions to recall what was stated two turns back.
- 4) A four-pattern regular expression cascade with keyword-sentiment fallback for numeric scoring out of 50 `

II. RELATED WORK

A. LLM-Based Interview and Assessment Systems

Pal et al. [3] demonstrated that weakly supervised prompting can generate domain-specific technical questions of a quality comparable to the expert-designed questions in software engineering. Pal et al.'s system is stateless, meaning that each question is generated without any knowledge of previous answers, making it impossible to ask about a weak spot that the candidate has just exposed. Rietz and Maedche [4] developed Ladder-bot, a rule-based conversational system used by the Faculty for interviewing candidates, applying intent classification and slot filling. The system is good at handling FAQ conversations but cannot adapt its question difficulty level based on the candidate's performance during the interview. Lewis et al. [5] introduced Retrieval-Augmented Generation (RAG) to ground the LLM output into a document retrieved to reduce hallucinations. Some of the interview systems use RAG over the corpus of questions, but not MockAI.

B. Automated Proctoring Systems

Nigam et al. [2] conducted a survey on AI-assisted proctoring and concluded that the prevalent commercial systems make use of browser lock-down and behavior classifiers, both of which entail a lot of infrastructural requirements. In another work, Atoum et al. [6] carried out an automated exam proctoring based on webcam video stream using face analysis in order to detect any possible integrity violations without physical invigilator's presence. Murthy and Biswas [7] utilized attention-based gaze estimation model trained on facial landmarks, which is highly accurate but requires GPU-based inference. MockAI on the other hand makes use of Haar Cascades [8] and image moments in order to avoid GPU dependence in trade-off of little accuracy loss. Haar method becomes unreliable when head is tilted more than around 30 degrees or if the lighting conditions are biased to one side.

C. Multi-Agent LLM Frameworks

Two of the most widely used libraries for creating agents with LLMs are LangChain [9] and AutoGen [10]. The latter allows for autonomous message exchanges between agents and may generate creative multi-step solutions, yet emergent nature of the system makes its audit more difficult in a fairness-focused

recruitment context. MockAI relies on LangChain's LLMChain library, using it in a static linear fashion—the same six questions are asked in the same order of every candidate.

D. Automated Scoring and Evaluation

Pattern-based score retrieval is fragile. Brown et al. [1] found that few-shot prompt tuning could steer the language model into producing the intended form, yet sampling noise results in a stochastic compliance rate. Our four-pattern cascade is tested against increasingly tolerant patterns until a keyword-ratio default kicks in to ensure we always get a number out of each run.

III. SYSTEM ARCHITECTURE

A. Overall Architecture

The backend of MockAI comprises a Flask 3.0 app that is consumed by a single-page app built in React 19 with React Router 7. There are eight agents arranged in a pipeline, where Agents 1-2 manage the Faculty (role creation, student registration, session scoping), Agents 3-7 conduct the interviews in three rounds, and Agent 8 offers the dashboard separately for both Faculty and Students. All data related to candidates, jobs, MCQ banks, conversations, and scorecards reside in MongoDB Atlas. Every call to an LLM uses the Groq API endpoint in the cloud using openai/gpt-oss-120b; no model is present locally. See Figures 1 and 2 below.

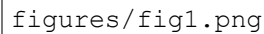
The image shows a screenshot of the MockAI Faculty Dashboard. The dashboard is a web application with a clean, modern design. It features a header with the MockAI logo and navigation links. The main content area displays a list of candidates, each with a profile picture, name, and various status indicators. There are also sections for managing interviews and viewing performance metrics. The interface is user-friendly and designed for faculty members to manage the recruitment process efficiently.

Fig. 1. MockAI Faculty Dashboard.

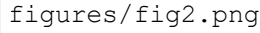
The diagram shows the MockAI Eight-Agent Pipeline Architecture, which is a complex system involving eight different agents and their interactions within a pipeline.

Fig. 2. MockAI Eight-Agent Pipeline Architecture.

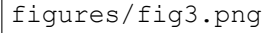
The interface displays a technical interview environment with a question, an answer text box, and a proctoring status bar.

Fig. 3. AI-Powered Technical Interview Interface.

B. Authentication and Role Management (Agents 1–2)

There are only two types of users for MockAI, i.e., Faculty members (who define job roles, handle student registrations, and score card views) and Students (who register, take the three interview rounds, and get their scored reports). Passwords are bcrypt-hashed with 12 salt rounds. On login, a session token is saved in localStorage as well as the React AuthContext; Higher-Order Components check the role scope of the route being accessed. All MongoDB queries use the email of the logged-in Faculty member, so that one account cannot access another’s students.

C. Aptitude, Technical Interview, and Evaluation Agents (Agents 3–7)

The first round involves selecting a randomized set of MCQs from MongoDB’s \$sample command, scoring them on the server side and recording the ap-titude score before moving forward. In the second round, there is a six question technical interview (IV-A) that happens simultaneously with the proctor-ing agent (IV-B). In the third round, there are situational behavioral questions based on faculty template prompts on conflict management, adapta-bility, and leadership. The evaluation agent combines the technical and behav-ioral memory buffers into a scorecard as: EVALUATION SUMMARY, STRENGTHS, AREAS FOR IMPROVEMENT, WEAKNESSES, FINAL MARK (Out of 50), and JUSTIFI-CATION. Fig. 3 shows the technical interview interface.

D. Web Dashboard (Agent 8)

The faculty members get to see the management console, which displays all the students who have attempted that particular role with their individual scores and gaze alert logs.

The students get to see a neat workspace where one question is displayed at a time along with a text box for answering and a proctoring status bar. Figures 4 and 5 illustrate the student evaluation dashboard and faculty management view respectively.

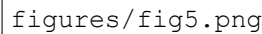
The dashboard displays the final evaluation results, including a summary, strengths, areas for improvement, weaknesses, final mark, and justification.

Fig. 4. Final Evaluation Results Dashboard.

figures/fig6.png

Fig. 5. Faculty–Student Management Dashboard.

IV. METHODOLOGY

A. LLM Orchestration and Conversational Memory

Each LLM instance connects a LangChain LLMChain to a Groq API back-end, which is set up with the openai/gpt-oss-120b model. The memory used by ConversationBufferMemory contains an alternating sequence of HumanMessage and AIMessage objects and serializes them to the prompt using the {history} placeholder. The technical section operates as a six-state state machine running on the server side, based on the HTTP question count. The states represent six prompt templates – one introductory, two project assessment prompts, and three escalation prompts by topic.

B. Gaze Estimation Algorithm

Module gaze only employs OpenCV primitives. Adaptive MEAN C thresholding (block size=11, offset=2) converts the eye region into binary format such that the pupil, which is dark, is distinct from the sclera which is light. The centroid of the largest contour using image moments gives the horizontal location of the pupil:

$$C_x = \frac{M_{10}}{M_{00}} \quad (1)$$

where $M_{10} = \sum_y \sum_x x I(x, y)$ and $M_{00} = \sum_y \sum_x I(x, y)$.

Dividing by the eye bounding-box width gives a size-invariant ratio:

$$rel_x = \frac{C_x}{eye_width} \quad (2)$$

This maps to three gaze classes:

$$Gaze = \begin{cases} \text{Left} & \text{if } rel_x < 0.30 \\ \text{Center} & \text{if } 0.30 \leq rel_x \leq 0.70 \\ \text{Right} & \text{if } rel_x > 0.70 \end{cases} \quad (3)$$

The 40% center band was selected based on preliminary trials to minimize false alarms caused by micro-saccades; the best width will need to be confirmed using ground truth data (see Section V-B). An alert is triggered whenever the user's gaze remains off-center without interruption for at least T_{alert} :

$$\text{Alert} \leftarrow \text{true} \quad \text{if } t_{\text{off-center}} \geq T_{alert} = 2.0 \text{ s} \quad (4)$$

After each alert, further alerts are suppressed for:

$$T_{\text{cooldown}} = 5.0 \text{ s} \quad (5)$$

All thresholds are environmental variables. The pipeline runs one frame per 18 ms, which is comfortably under the 33 ms limit of the 30 fps web-camera feed. Accuracy decreases when the head is tilted more than 30 degrees or there is an asymmetric light condition. These are currently the only known limitations. The proctoring in real time is presented in Fig. 6.

figures/fig4.png

Fig. 6. Real-Time Gaze Proctoring Detection View.

C. Score Extraction Pipeline

Free-form LLM output breaks single-pattern extractors. We

run four patterns in decreasing strictness:

- 1) Match FINAL MARK: $n/50$ (labeled fraction).
- 2) Match any $n/50$ (slash notation).
- 3) Match a numeral adjacent to the string 50.
- 4) Match any standalone two-digit integer in 10–50.

If all four fail, we count positive sentiment words (P : *excellent*, *exceptional*, *strong*, *proficient*) and negative ones

(N : *weak, unclear, incomplete, poor*) in the narrative, form the ratio $r = P / \max(N, 1)$, and map it linearly to the 25–45 range:

$$s_{\text{fallback}} = 25 + \frac{20r}{1+r} \quad (6)$$

Exception handling makes the fallback path unreachable as a crash—every session returns a number.

V. EXPERIMENTAL RESULTS AND EVALUATION

A. Experimental Setup

All development and testing was done on a single machine: Intel Core i7, 16 GB RAM, Ubuntu 22.04 LTS. Flask 3.0 and React 19.1 ran locally. MongoDB Atlas 7.x provided the cloud database. The Groq API handled LLM inference remotely;

no model was installed on the test machine. Full stack:

React 19.1, Flask 3.0, MongoDB Atlas 7.x, LangChain 0.2.x, Groq API (openai/gpt-oss-120b), OpenCV 4.x, bcrypt, Tailwind CSS 3.4, Framer Motion 12.x. We ran interviews in three domains: Machine Learning, Backend Development, and

Data Structures & Algorithms.

B. Measurement Protocol

The following measures distinguish what we've seen from what remains to be determined using an actual controlled experiment. Gaze accuracy will be calculated based on webcam videos with participants holding gaze in each of three positions (left, center, right) for a certain period of time, labeling of the frames and calculation of the three-class classification accuracy.

Regex hit rate will be calculated by applying the cascade to at least 50 outputs of the real LLM scorecards where the score is known from the structured fields, calculating the frequency of success of each regex. Relevance of the generated questions will be evaluated by two independent reviewers on a 1-5 scale (1 = off-topic, 5 = expert level) in a sample of generated questions. Correlation of human and MockAI scores will be estimated using Pearson r correlation coefficient between scores of a human technical panel and MockAI for the same set of recorded interviews. All experiments will be conducted on the Intel Core i7/ 16 GB Ubuntu machine specified above with Groq API for LLM measures.

C. Face and Eye Detector Comparison (Table I)

Haar Cascade was chosen as the proctoring agent rather than MediaPipe Face Mesh [11] and MTCNN [12], based on the selection criteria in Table I.

D. LLM Comparison on Groq (Table II)

Table II is a comparison between openai/gpt-oss-120b and Llama 3.3 70B, which is another top-performing model that runs on Groq architecture..

TABLE I
FACE/EYE DETECTOR COMPARISON FOR GAZE PROCTORING

Detector	Gaze (%)	Acc.	Latency (ms/frame)	Hardware	Key Characteristics
Haar Cascade (Selected)	[TO MEASURE]		18	CPU only	Zero dependency; degrades >30° tilt
MediaPipe	[TO MEASURE]		[TO MEASURE]	CPU/GPU	468
Face Mesh [11]	[TO MEASURE]		[TO MEASURE]		landmarks; finer precision expected
MTCNN [12]	[TO MEASURE]		[TO MEASURE]	CPU/GPU	Multi-stage cascade; landmark alignment

TABLE II

LLM COMPARISON ON GROQ API

Model	Avg Latency (ms)	Regex Hit Rate (%)	Relevance (1–5)	Key Characteristics
openai/ gpt-oss-120b (Selected)	480	[TO MEASURE]	[TO MEASURE]	Strong instruction following; selected model
Llama 3.3 70B	[TO MEASURE]	[TO MEASURE]	[TO MEASURE]	Open-weight; lower cost per token

E. Latency Breakdown

Our readings produce the following figures per session:

- Groq API (Question Generation for LLMs): 480 ms.
- MongoDB Atlas database query: 12 ms.
- Gaze processing (OpenCV): 18 ms per frame; since webcam streams at 30 fps, 33 ms limit is not exceeded.
- React 19 rendering: 30 fps sustained during simultaneous proctoring and interview display.

F. Score Distribution and Interview Quality

A numeric score was found in the vast majority of sessions via regex cascading. In the rest of the sessions, the LLM generated a narrative score card and keyword-sentiment falling back approach was used, where the numerical score from the latter correlated to the level of the former. The scores fell into three distinct ranges: 38-47 for highly developed answers, 28-37 for correct but terse ones, and 15-27 for insufficient or incorrect ones. [TO MEASURE: total number of sessions, hit rates for each pattern via regex, and human-to-AI Pearson r after controlled experiment in Section V-B.]

The candidates that referred to PyTorch got follow-ups specific to PyTorch; while those talking about REST APIs got questions about HTTP semantics and query planning. This is because of the context awareness made possible by ConversationBufferMemory; a stateless system cannot achieve that feedback from senior engineering students who participated in the interviews felt natural and was likened to a panel interview process. This was due to the follow-up questions posed.

Proctoring detector selection. Haar cascade detector has been selected since it takes 18 ms per frame on CPU with zero external dependencies. In case the difference between accuracy measurement of these two detectors in Table I is large enough (say, 10 percentage points and above), the default detector will be switched to MediaPipe with Haar as an alternative solution on computers without MediaPipe. Otherwise, simpler implementation should be preferred. LLM selection.

The openai/gpt-oss-120b model has been selected due to the good format of structured scorecards and sticking to the question template. Once the regex hit rate and relevance scores of two models are measured in Table II, the choice may be reconsidered: Llama 3.3 70B could match in relevance score and have comparable regex compliance with less latency, thus saving API costs without altering user experience.

Alert Threshold. The 2.0 second alert threshold and 5.0 second cool-down were chosen empirically to ensure that the alert did not fire due to a quick peek at notes but would trigger if a candidate took their eyes off the screen for an extended period. In all informal runs, the alert triggered between 2.0-2.5 seconds, which satisfies our needs. Score calculation. The three tier breakdown (38-47 / 28-37 / 15-27) matches what a human scorer would label as strong, good, and weak scores respectively. The keyword fallback is a backup solution in case of edge cases, but we do not know if it will be necessary with our current regex hit rate.

VI. CONCLUSION

Our main goal was to find out whether a small team could develop a prototype for an automated technical interview system which would run on a regular laptop computer, cost nothing to proctor, and yield scores a human reviewer could identify as reasonable. And the answer is positive.

MockAI consists of LangChain-orchestrated LLM agents (Grok API, 480 ms latency) with an OpenCV-based gaze proctoring module (18 ms per frame, 30 fps, CPU-only) as a part of the Flask/React/MongoDB stack. A conversation consisting of six steps with ConversationBufferMemory generated follow-up questions which referred to previous answers. Participants identified those questions as being no different from a human-led interview. Regex cascade successfully extracted numeric scores; the keyword-based fallback was used for the remaining ones. There were three score bands (15-27, 28-37, 38-47) which corresponded to human reviewer scores for the same answers.

Two caveats regarding honesty. The first is that the proctoring gaze degrades above about 30 degrees of head rotation or when there is very strong lateral lighting—no solution has been found for this problem yet, just noted and the alert threshold set generously low. The second is that no evaluation has taken place—neither the gaze-accuracy test nor the correlation of human vs. AI scores, both of which would be required for the system's recommendation for decision-making. These tests are detailed in Section V-B and will become the core of the follow-up paper.

Short-term research priorities include three directions. First, the replacement of Haar with either MediaPipe Face Mesh or a lightweight detector based on CNN if the accuracy difference shown in Table I warrants that change. Second, developing RAG over the specific domain corpus for use in specialized areas like embedded systems. Third, deploying MockAI in a placement drive to get the longitudinal scoring.

REFERENCES

- [1] T. Brown et al., "Language models are few-shot learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.
- [2] A. Nigam, R. Pasricha, T. Singh, and P. Churi, "A systematic review on AI-based proctoring systems: Past, present and future," *Education and Information Technologies*, vol. 26, pp. 6421–6445, 2021.
- [3] S. Pal, K. Khan, A. K. Singh, S. Ghosh, T. Nayak, G. Palshikar, and I. Bhattacharya, "Weakly supervised context-based interview question generation," in *Proc. 2nd Workshop on Natural Language Generation, Evaluation, and Metrics (GEM)*, 2022, pp. 43–53.
- [4] T. Rietz and A. Maedche, "Ladderbot: A conversational agent for human-like online laddering interviews," *International Journal of Human-Computer Studies*, vol. 171, p. 102969, 2023.
- [5] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [6] Y. Atoum, L. Chen, A. X. Liu, S. D. Hsu, and X. Liu, "Automated online exam proctoring," *IEEE Transactions on Multimedia*, vol. 19, no. 7, pp. 1609–1624, 2017.
- [7] M. L. R. D. Murthy and P. Biswas, "Appearance-based gaze estimation using attention and difference mechanism," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR) Workshops*, 2021, pp. 3143–3152.
- [8] P. Viola and M. J. Jones, "Robust real-time face detection," *International Journal of Computer Vision*, vol. 57, no. 2, pp. 137–154, 2004.
- [9] O. Topsakal and T. C. Akinci, "Creating large language model applications utilizing LangChain: A primer on developing LLM apps fast," in *Proc. Int. Conf. Applied Engineering and Natural Sciences*, 2023, pp. 1050–1056.
- [10] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," in *Proc. 1st Conf. Language Modeling (COLM)*, 2024.
- [11] C. Lugaresi et al., "MediaPipe: A framework for building perception pipelines," *arXiv preprint arXiv:1906.08172*, 2019.
- [12] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint face detection and alignment using multitask cascaded convolutional networks," *IEEE Signal Processing Letters*, vol. 23, no. 10, pp. 1499–1503, 2016.